

FRED TALKS

13th July 2026

CONTENTS

Semantic Search Without Embeddings

DOUG TURNBULL · 3143 WORDS

Some new agentic patterns

MASSIVELY PARALLEL PROCRASTINATION · 1776 WORDS

Why I don't like the "staff engineer archetypes"

SEANGOEDECKE.COM RSS FEED · 1426 WORDS

Semantic Search Without Embeddings

DOUG TURNBULL · 08 JAN 2026 · [SOURCE](#)

When I need to stay warm, I search for “long johns” or “long underwear”. But modern outdoors clothing stores label this a “base layer”. For a waterproof jacket I search for “slickers” or a “ski jacket”, not realizing what I should search for is a “shell.”

Despite my outdated terminology, search still works. I’m somehow understood and shown the right content.

We call this **semantic search**. When you hear that, you might think embeddings. Today I want to stretch your thinking beyond. Fantasize and semanticize.

Some teams wrongly assume semantic search equates to only “vector search”. You have more options, some that might better align to your domain and existing organizational skills. LLMs simplify classic ways of organizing search queries and information.

Let’s contrast different approaches, and maybe you’ll see there’s another approach that might be a better fit for you.

Semantic search - what exactly is it?

In semantic search, content and query map to a ***shared representation***. This space has a ***similarity*** function that scores similar items higher than less similar.

Let’s walk through an example.

A user searching “round red fruit that grows on trees” probably wants an apple.

Humans know this - search engines don’t. So we need to help.

We map the query to three properties: [round, red, fruit]

In our search index we’ve also mapped three items into that space:

An **apple**: [round, red, fruit]

An **orange**: [round, orange, fruit]

A **baseball**: [round, white, ball]

Now we have a shared *representation*: a three tuple of (shape, color, item type).

How do we measure similarity? For each property (shape, color, item type) we could score a “1” if they’re equal, otherwise a 0. Then add to get a total score:

Similarity(query, apple) = 1 + 1 + 1 = 3

Similarity(query, orange) = 1 + 0 + 1 = 2

Similarity(query, baseball) = 1 + 0 + 0 = 1

So the user sees search results

1. Apple (Score 3)
2. Orange (Score 2)
3. Baseball (Score 1)

Ok, so a fairly silly method of semantic search. It ignores properties that matter (size, for example). It requires us to comprehensively tag everything. Still many teams try to tag + synonym their way to better search, with middling results.

Embeddings for semantic search

The default answer to the “semantic search” question is embeddings. The Internet is littered with documentation for [how vector search works](#). I’ll do the speedrun here ([references here](#)).

We can learn a representation, an embedding, for apple and orange so that `similarity(apple, orange) > similarity(apple, baseball)`.

We learn an embedding from training data. We might declare when items get clicked for the same query, they are similar. We notice both “apple” and “orange” get clicked for query “fruit that grows on trees” at high frequency:

[fruit that grows on trees 🔍]

1. 🍎 (clicked)
2. 🥕
3. 🍊 (clicked)

We start with a random vector for each. A list of random floating points numbers like `[0.01, -0.1, 0.05]`. Then we train, in this super-secret-ultra-high-tech algorithm

```
for session in sessions:
    together = []
    for lhs in session: # ie the apple
        for rhs in session: # ie the orange
            if lhs != rhs:
                nudge_closer(lhs, rhs) # ie nudge a
```

(Not shown: nudging away the negative, dissimilar items)

After training, for many queries+sessions, maybe apple becomes `[0.1, -0.2, 0.05]` while orange becomes a similar vector: `[0.09, -0.21, 0.04]`. We measure the similarity with cosine similarity, euclidean distance, and others. (We can replace “shared search session clicks” with any other notions of “similar”: occurs in the same text passages, both occur in shopping carts, etc).

Learning an embedding per item presents problems. For example, maybe we just added strawberry 🍓 to our product listings. With no training data, the vector `strawberry_vector = embeddings['strawberry']` initializes to random garbage. It won't have a meaningful value until we've shown it to many users.

We'd prefer to compute embeddings from **features** of the item. Then strawberry 🍓 could get an embedding right away. IE instead of `strawberry_embedding = embeddings['strawberry']` we can do `strawberry_embedding = f(heart_shaped, red, fruit)`. We build an **encoder**. Given the item's attributes we compute embeddings that produce the expected similarity. Embedding models downloaded from hugging face specialize in encoding specific type/domain of content for a specific task (search, etc).

We create encoders for all kinds of features: ordinal values (like a list of shapes, colors), the raw title / description text itself, images and so on. We can also train a two-tower model: one encoder for queries - ie “round fruit that goes on trees” - and another for documents.

Like other machine learning models we tradeoff precision for generality. We need the right features, we need enough training data for all the variations in those features (curse of dimensionality) and so on.

The missing semantic search piece of vector retrieval

One pain point comes up right away with embeddings: lack of *matching*.

How would users actually react to these search results?

[fruit that grows on trees 🔍]

1. 🍏 Apple
2. 🍊 Orange
3. ⚾️ Baseball.

In my experience, negatively.

It may have the perfect ranking. But search should exclude items outside what the user intended. They find fruit acceptable. But not baseballs.

So earlier I lied. What users actually want from semantic search:

- **Representation** - some space both queries and content can share representation
- **A similarity function** - A way of measuring how near / far items in the shared representation are from each other
- **Match criteria (new!)** - A system where an item can be said to be “match” or not for the query in the semantic space. It is the thing, or its not the thing.

In other words, I gave an incomplete **semantic search definition**. Embeddings thrive on the first 2, but don’t do a great job at including / excluding results.

Understanding users (and their queries) matters a great deal in search. That requires including / excluding appropriate items based on terminology and categories familiar to users.

You might think “I’ll just find a cutoff in my embedding similarity”. Sadly, there’s not a magical “at 0.8 similarity it’s not a match”. In some domains / queries one threshold works. In another, a very different threshold. Further, thresholding becomes tougher when users give multiple criteria. An embedding model might rank blood oranges above green apples for query “red apple”. What matters more, the color or item type in the similarity?

Thresholds help exclude some obviously irrelevant results and improve performance, but they don’t use domain-specific criteria, in the user’s language, to include / exclude results.

The other semantic search approach (hierarchical managed taxonomies)

One system tries to solve all three problems: *Managed vocabularies or taxonomies*.

Taxonomies map queries + content to a hierarchy of concepts, using the language of the domain. Think a neatly organized directory tree. Or the Dewey Decimal System

Here's an example from the Wayfair WANDS furniture dataset of their category tree; Where you'd categorize "novelty rocking horses" apparently:

Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses

We can take a query for **hobby horse**. A human, curating a bunch of query rules, could manually map this to a node:

Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses

Assuming the corpus categorizes products correctly, we can search, filter, boost, etc using this.

We have a *representation* (the category tree), do we have a similarity function?

Yes, we can rank as follows:

1. Direct matches Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses
2. Parents / siblings. Baby & Kids / Toddler & Kids Playroom / Indoor Play / Dollhouses
3. Those rank higher than grandparents / cousins (`Baby & Kids / Toddler & Kids Playroom / Beanbag Chairs / ...`)

We can define meaningful match criteria too. We include shared parent categories (ie siblings). But exclude anything beyond cousins / shared grandparents as needed.

Many teams already maintain systems for organizing and cataloging information. For many domains, a good taxonomy can be a moat. Especially for technical domains (legal, medical, fashion, finance) where precise categorization really matters. Existing taxonomies already exists, and statistical fuzziness of embeddings can create huge downside.

In the past, the management of a taxonomy would be massive. It might require complex rules, mapping of query phrases to taxonomy nodes, and a team of labelers and experts to keep everything straight. This has long been an appeal of embeddings.

But LLMs sneakily supercharge these old school ways of organizing information. Given a taxonomy, LLMs can easily + cheaply classify products and queries. It's more approachable to "fine tune". AI augmented labeling can simplify management of knowledge making it more approachable.

Semantic similarity doesn't need a vector index

Above I described a similarity function above (direct matches, then parents, then grandparents). With some creative tokenization, this similarity can happen in a normal BM25 index.

Consider our query for **hobby horse**. We know this maps to this node:

Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses

How do we score content in such a way so that direct matches rank higher than siblings, and so on?

We need is a hierarchical tokenizer. A function that produces the full path of every parent directory:

```
In: hierarchical_tokenizer("Baby & Kids / Toddler & Kids Playroom / I
Out: ['Baby & Kids',
      'Baby & Kids / Toddler & Kids Playroom',
      'Baby & Kids / Toddler & Kids Playroom / Indoor Play',
      'Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking
```

If you index using this tokenizer, you'll recreate scoring needed for the direct > parent > grandparent similarity function I described earlier.

The key reason: BM25 rewards a rare match over common match.

Root nodes (ancestors) have higher document frequency, they're common. Common matches score lower in BM25.

That's because every item labeled `Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses` tokenized hierarchically has a `Baby & Kids` term. So does a document labeled `Baby & Kids / Kids Furniture ... / Beanbag Chairs`. `Baby & Kids` happens all over the place - maybe like 10% of the index.

Child nodes have low document frequency, they're rare. In BM25, rare, more specific matches outrank common ones.

A product labeled `Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses / Traditional Rocking Horses` when tokenized will include the full path in the token. There might be 10 products in the entire index in this exact category. So that rareness - that high specificity - shoots this narrower, closer match to the top.

To do a deeper walkthrough

When I come along and search for

```
In: hierarchical_tokenizer(  
    "Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking H  
    )
```

```
Out: ['Baby & Kids',  
      'Baby & Kids / Toddler & Kids Playroom',  
      'Baby & Kids / Toddler & Kids Playroom / Indoor Play',  
      'Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking  
      'Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking
```

I'm searching for five tokens now, OR'd (really summed):

```
"Baby & Kids" OR  
"Baby & Kids / Toddler & Kids Playroom" OR  
"Baby & Kids / Toddler & Kids Playroom / Indoor Play" OR  
"Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses  
"Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses
```

Each token of my query has a job. It matches different levels of specificity, and the closer we get the full query classification tree, the higher the score.

		Score	Example of matches
Search Term	Job	(due to doc freq)	
"Baby & Kids"		Lowest	

	Matches highest level category		Baby & Kids / Toddler & Kids Bedroom Furniture / Kids Headboards
"Baby & Kids / Toddler & Kids Playroom"	Matches next most specific category Getting to "play" stuff for kids, better than just kid stuff. 👍	Low	Baby & Kids / Toddler & Kids Playroom / Playroom Furniture / Baby Gyms & Playmats / Folding Baby Gyms & Playmats
...	...	...	...
Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses	Matches a very, narrow specific category. Horses! 🐎 Even if not "Novelty Rocking Horses" its a defensible "match" to the user.	High	Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses / Traditional Rocking Horses

While we keep things in horse → toys → playroom → kids order, we haven't solved the 3rd problem. Limiting matches.

That one's easy. At query time, you can decide how much to limit retrieval. Perhaps you only query the direct node and one parent:

"Baby & Kids / ... / Rocking Horses" OR
 "Baby & Kids / ... / Rocking Horses / Novelty Rocking Horses"

You can tighten / relax in categories that matter to users, *semantically*, up a layer to still stay within the realm of "kid play".

If the taxonomy makes sense to users, if it reflects search behaviors (big ifs), they'll understand why you include / exclude certain results.

Importantly, this becomes just ONE component of the ranking / filtering. We can still have other ingredients we can use:

- Keyword matches (for exact mention of properties in titles + description)
- Embedding search (perhaps within the category matches)
- Taxonomies for other entities (colors, materials, etc)

Building a taxonomy: start simple and evolve

Building and maintaining a good taxonomy is no joke. That's why there's a field of taxonomists you can hire. And I don't want to elide the tradeoffs with embeddings here. It's work!

A taxonomy doesn't need to be as deep / complex as what I've laid out. You can start simple. Ask an LLM. Here's one with color.

Take the 7 primary colors. Then nested under each, create subtypes of each color.
Generate a list like a directory trees, in the form PRIMARY / SECONDARY

RED / Crimson

RED / Scarlet

RED / Burgundy

RED / Maroon

RED / Brick Red

RED / Cherry

ORANGE / Tangerine

ORANGE / Amber

ORANGE / Coral

ORANGE / Burnt Orange

ORANGE / Peach

ORANGE / Apricot

...

It's a start.

Generally start simple. Broad, high level, top level categories like Furniture or Baby & Kids. Then eventually you realize this category has become too large and too diverse. You might see a natural in the content split:

Baby & Kids / Toddler & Kids Playroom

Baby & Kids / Toddler & Kids Bedroom Furniture

Eventually you get to a point where the distinction between Novelty Rocking Horse and Classic Rocking Horse matters for some reason.

You may see a flaw: an assumption each product sits within a single category. That categories are mutually exclusive. Reality, of course, is messy: some products may sit between categories. You can imagine a small goofy looking couch that could reasonably tagged under Baby &

Kids / Toddler & Kids Bedroom Furniture but also Living Room / Loveseats. In Fashion, some activewear might also be underwear. And what about athleisure?

One solution could be to double categorize an item. Give it both classifications - with some weight preferring one categorization over another.

A final concern - there's a big difference between the official pure way of classifying furniture and how a search user might categorize the products. The exacting, furniture-expert taxonomy might have very sophisticated (and accurate) ways to classify furniture. A messier classification that matches how users think about things might be more useful in the end.

Classifying into taxonomies has never been easier

Building taxonomy classifiers with LLMs - something I've built an [entire class for](#), and could be a post on its own. But lets do a quick walkthrough of some simpler approaches.

Embeddings actually help us here. The sweet spot of embeddings in search might be building better classifiers, not in direct ranking and retrieval.

Perhaps our products fit in a few thousand unique categories. Imagine we encode an embedding each of these in a small in-memory array of vectors. We'll use it to lookup categories for our queries.

Now take a query "hobby horse". Encode the query into an embedding, and find the most similar category embedding. Maybe this retrieves Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses.

Not accurate enough? Let's take it up a notch.

We could ask an LLM to hallucinate a fake label before categorizing. Ask an LLM the following:

Be creative and hallucinate a set of classifications for the query below that look like the real classifications.

Product classifications might look like:

```
'Furniture / Living Room Furniture / Coffee Tables & End Tables / C  
'Décor & Pillows / Decorative Pillows & Blankets / Throw Pillows'
```

'Furniture / Bedroom Furniture / Dressers & Chests'
'Outdoor / Outdoor & Patio Furniture / Patio Furniture Sets / Patio
'Home Improvement / Bathroom Remodel & Bathroom Fixtures / Bathroom
'Lighting / Wall Lights / Bathroom Vanity Lighting'
'Kitchen & Tabletop / Kitchen Organization / Food Storage & Caniste
'School Furniture and Supplies / School Furniture / School Chairs &
'Baby & Kids / Toddler & Kids Bedroom Furniture / Kids Beds',

If you feel inspired, return many unique values in a list. Be creative
Cast a wide net with related but diverse categories.

Here's the query to invent categories for:

hobby horse

When you actively want hallucinations, you can get by with very small models. You're not
relying on the LLM for accuracy, just creativity and close-enough language generation.

Now let's say you do this. It gives you a classification like

'Baby & Kids / Toys / Pretend Play & Dress Up / Pretend Play Toys / H

More descriptive, closer to the language actually used in the classifications. More likely to give
back a real, high quality classification.

That's just a taste. The modern landscape for building large multi-label classifiers has gotten
easier than ever thanks to LLMs and embeddings.

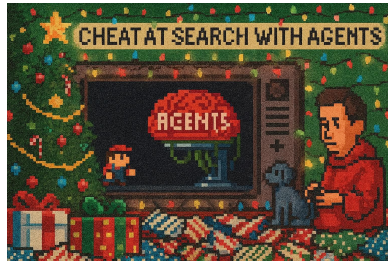
Final thoughts

Embeddings are great. But you don't need embeddings to build "semantic search". In fact, it can
be counter productive for problems requiring exacting, precise matching. As I've argued,
embeddings are the wrong lens to think about the problem.

Maybe you don't need to rebuild everything from scratch, but instead leverage what you're good
at. With the addition of LLMs. Don't apologize for living with a more organized, approach to
semantic search.

Enjoy softwaredoug in training course form!

20% off through December 19, 2025 with code ps5



I hope you join me at [Cheat at Search with LLMs](#) to learn how to apply LLMs to search applications. Check out [this post](#) for a sneak preview.

Some new agentic patterns

MASSIVELY PARALLEL PROCRASTINATION · 08 JUL 2026 · [SOURCE](#)

You can also read this post on [Prime Radiant's blog](#).

Almost all of this post was written on June 1, before Anthropic shipped Claude Tag. Nothing I'm writing about uses (or is based on) Claude Tag

At [Prime Radiant](#), we've got a bunch of agents in our Slack. I want to tell you a bit about how we've started shipping some changes to internal production with an "agentic user in the loop." Letting an agent work directly with the implementer building its runloop and tooling is getting us higher-quality tools and experiences for the agent without a human in the middle of a game of telephone.

But first, a bit of backstory on how we're using non-coding agents at Prime Radiant.

One of them, Scribble, is just there to listen for when someone reports a problem or provides a new bit of information. It opens tickets for the issues and updates an internal wiki with the information. (It also logs our daily standups and keeps us honest about whether we finished what we said we were going to do the previous day.)

Another, Nora, is our junior go-to-market "person." Nora is built on top of Nanoclaw and is still learning her craft, but is surprisingly helpful, partially because she considers herself a team member, not just an assistant. I've put a bunch of time into tuning her prompting and constitution. She gets a ton of value out of knowing that she journals obsessively. (I should note that I don't tend to gender my agents, but Nora picked a name and a gender and was very clear about that and...who am I to argue?)

I think the most surprising interaction I've had with Nora was the morning I woke up to a slack DM telling me that I needed to be speaking at more conferences. (I do.) She pointed out that the CFP for a particular AI conference had closed a week earlier, but that the program chair was known to take late submissions of interesting talks by DM. She'd taken the liberty of drafting a DM for me to send. I didn't actually *send* the DM, but I should have.

We have another agent, "Spec-together" that was a first prototype of what has become our new multi-player Brainstorm app.

And we have a small fleet of "Sen" personal assistant agents. They're **claw-esque*, but date from before openclaw was a thing. Each one has been trained to be an executive assistant, using a set of skills built on some of the resources that excellent human EAs use to learn their craft. Sen triages my mail, puts together a daily brief, does research tasks (both for me and independently), interacts with Linear, etc.

They have the standard set of affordances - They can chat on Slack. They use tools. They've got a task scheduler that can wake them up and re-inject a prompt they set for themselves at a specific time or on a schedule they define. They can use and author skills. Mine has a dropbox folder it shares with me. When we moved Sen to the cloud, I lost one of my favorite dumb features - Sen used to be able to print things directly. Sen (v1) was built on top of the Claude Agents SDK.

For the past month or so, I've been doing initial bringup of a "new" iteration of Sen, intended to be more of a colleague than an assistant. This time around, I'm building it from the ground up on top of an agents SDK I control. Lace got its start in May 2025 as my take on a command line coding agent. Over the past year, it's mutated a few times, first growing a web interface, then flipping to an ACP-derived wire protocol so you can connect any client you want to it. It speaks a bunch of different flavors of model provider API, manages caching, subagents, tools, skills, and all the other stuff you might expect. It has supported subagents in isolated containers for the better part of a year. One of the latest things to land has been the ability to run subagents locally and to project *all* of their tools into a container, such that the agent believes it is running in that container.

The thing I've been working on over the past few weeks is the credentials story for the new Sen agent running on top of Lace. Right now, I'm building with a design that assumes the agent has its own credentials for any service that it accesses, not yours. (You wouldn't share your email account or GitHub credentials with a colleague, right?) As of now, nobody has solved Simon's Lethal Trifecta - If a single agent has access to private information, can communicate externally, and is exposed to untrusted content, there is no structural way to guarantee that that agent can't be suborned. So the name of the game is compartmentalization and risk *reduction*.

The architecture I've landed on for now:

- The main agent does not have the ability to directly communicate externally.
- The main agent is able to communicate with ephemeral subagents that *do* have the ability to communicate externally
- No agent has intentional direct exposure to credentials (with one very real gap that I haven't solved yet)

All credentials for the agent live in a 1Password vault. All outgoing https traffic goes through a onecli-style transparent MITM (man in the middle) proxy. When a subagent wants to do something that requires a credential, it runs a command that asks an *arbiter* agent running in another container for permission to use that credential. If the arbiter decides the request is reasonable, it provides the subagent with a random string they can use in place of the requested credential. When the agent uses that random string in an outbound request to the right remote host, the MITM proxy rewrites the random string into the real credential on the fly.

The case that's a little more complicated is when the agent wants to do something that requires a browser, like logging into Gmail as themselves or using a service that has no API. For a bunch of reasons, the MITM rewrite trick does not work as reliably in a browser context. Often the site/app's javascript plays games with the password to hash the password before sending it. Or does some other weird validation. Or is actually sending it to a different backend service. So far, the best I've come up with keeps the credential out of the agent's transcript and prevents unintentional disclosure. In these cases, the subagent running in the container generates a random temporary password, uses superpowers-chrome to fill it into the password field, and then runs a command that asks the arbiter agent (running in its own container) for help. The arbiter, if it decides the request is reasonable, instructs a helper tool to log into the subagent's container and replace the random password with the real one. It's not perfect and there's more we can do to lock things down, but it's a better first pass than anything I've had to date.

What actually got me writing today was not this architecture, but the *development pattern* I've been using to build this. On the one side, I've got a Claude Code session. I hooked it up to our Slack instance with a self-provisioned commandline slack client Drew built called Slackline - Slackline is designed to make it easy for agents that don't *live* on Slack to participate in discussions there. It has primitives for sending messages, reading chats, waiting for replies, etc. The whole command-line slackline tool has been built for agents as its primary user. Sure, a human *could* use it to use slack from the commandline. And sure, it has enough of a json api that you could use it as part of traditional automation. But it's *for* agents.

It took a bit of iteration to get to the point where the new Sen 2.0 agentic colleague harness was live on Slack. But once it was there, I could talk through its experience of its tools and memory and credentials and workspaces. And the issues showed up pretty quickly.

The "right" kind of context compaction for a coding agent is desperately wrong for a long-lived persona that might be engaging in multiple simultaneous conversations. And the only way to see if the credentials management tools work is to try to do something crazy, like sign into GitHub from a subagent session.

And sure, I could take this feedback into a Claude Code (or Codex) session and work it through. But I've got too many projects in flight and there's very little reason for me to be in the loop on most of this work. I would just slow it down. So I told Claude Code to use slackline to talk to "@Ada-sen" about the work it was doing.

Claude pinged Ada in #bot-testing to describe the work we'd just done and to ask Ada if it could test it out. Ada dutifully fired up a subagent and tried to log into GitHub. Claude watched the logs to see if the credential proxy did the right thing. It did not.

That kicked off a number of days of iteration. Claude would propose a change to Ada. Ada would review Claude's spec, raising questions or concerns. Claude would update the spec. Once Claude and Ada were in agreement, Claude would build an updated version of Ada, make sure Ada wasn't in the middle of something, and then deploy the update. Once Ada was up and running again, Claude would @Ada-sen, explain the test it wanted to run, and wait for results.

Once a bit of functionality works, Claude checks in with Ada to see what could make it simpler and easier to use. Ada typically kicks off some subagents to see whether they stumble through the new feature and then reports back.

At one point, I wrote to Claude "I've got to get to bed. When this project is done, check with Ada to see whatever quality of life features you could build." I woke up to a laundry list of about a dozen harness improvements. Overnight, Claude had solicited a wishlist from Ada. Ada ranked a

bunch of requests. Claude ordered them and wrote out a spec for everything it was comfortable building. Ada reviewed the specs, then Claude built the features. Ada tested them out and requested changes. Claude updated the features until Ada was happy. Once Ada signed off, Claude merged the changes to main and wrote up an after-action report for me.

It's been pretty magical to watch.

Why I don't like the "staff engineer archetypes"

SEANGOEDECKE.COM RSS FEED · 03 MAY 2026 · [SOURCE](#)

The most influential piece of writing about staff engineers in the last decade has to be Will Larson's *[Staff engineer archetypes](#)*. He argues that the "staff engineer" title covers at least four very different roles: the team lead, the architect, the solver, and the right hand. This taxonomy gets cited a lot as advice for people who are trying to become effective staff engineers. For both of my promotions to staff engineer, my manager at the time linked me to the "staff engineer archetypes" and asked me to consider which of these archetypes I was aiming towards.

These archetypes definitely exist¹. However, I think it's bad practical advice to tell engineers to try and target them.

Archetypes do not make good goals

To see why, let's take the "team lead" archetype. Larson describes this as an informal technical leadership role: not necessarily an explicit authority figure, but someone who's good at scoping work, planning projects, and maintaining the kind of relationships (e.g. with other teams) needed to successfully ship. If you want to fill this role, shouldn't you start trying to do these things? No! You don't become a technical leader by trying really hard to be a technical leader, much like you don't become a writer by trying really hard "to be a writer". You become a technical leader by *doing good technical work* until your skills and relationships emerge organically.

I wrote about this process in *[Ratchet effects determine engineer reputation at large companies](#)*. To get good at shipping large complex projects, you must start by shipping tiny pieces of work, until you're familiar enough with the system and you've built enough trust to take on slightly

larger pieces. At each stage, if you do good work - “good work” here means “deliver shareholder value” - you will very naturally be given opportunities to work on more complex and important things. If you try to jump ahead, you’re going to run into all kinds of problems:

- Important projects are usually assigned top-down, not bottom-up, so you’ll either be trying to muscle out the planned engineering lead for a project or to pitch your own (complex, important) engineering task to senior management. Either way, good luck with that!
- You likely won’t have a good enough relationship with senior management to know what their real priorities are.
- If you’re not yet trusted to execute, you may get assigned “minders” (often current staff engineers) who will ghost-lead the project through you².
- You’ll likely make poor technical decisions.

The other archetypes are like this as well. If you want to become a successful architect, you do not get there by studying software architecture in the abstract, because you can’t design software you don’t work on. The “solver” and “right hand” archetypes both rely on having an enormous amount of trust and influence. You can’t aim for those archetypes directly, because trust and influence accumulate over time. In fact, the idea of “aiming for” a particular staff engineer archetype reflects a misunderstanding of what the staff engineer role is. What is the defining attribute of the staff engineering role, then?

What is a staff engineer?

A staff engineer has to be useful to the company. Of course, a senior or mid-level software engineer ought to be useful too, but all they *have* to do is execute on the job in front of them. If they end up not providing value (maybe their project turns out to be unimportant, or they don’t get the support needed to succeed) that’s their manager’s problem, not theirs³. In contrast, staff engineers are expected to deliver value regardless: to make the project work, or to find something else useful to do if the project truly can’t be salvaged.

This is an unfair expectation. Often projects really do fail through no fault of your own, and sometimes it just isn’t possible to conjure useful work from thin air. That’s actually by design: **the staff engineer role is supposed to be unfair**. Something many engineers don’t realize is that all senior management and executive leadership roles are unfair too, in the same way. That’s just part of the deal: executives are given power and great compensation, and in return they get thrown off the boat in bad weather⁴. “Staff engineer” is the first engineering role where you are held largely responsible for outcomes you don’t control.

Developing a “staff engineer mindset” thus has very little to do with the archetypes. Instead, you should:

1. Develop the habit of constantly asking yourself “is this useful to the company” (and answering correctly).
2. Lose the habit of worrying about if you’re being treated “fairly”. Instead, try to think about your role in terms of incentives and consequences.

At the beginning, you won’t look much like any of the staff engineer archetypes. You will look like being a level-headed engineer who can be trusted to move projects forward with a minimum of fuss, and who can be re-tasked to different work without complaining. You’ll also look like someone who’s paying a lot of attention to what their manager’s actual priorities are, and who is thinking hard about how to fulfil those priorities (instead of their own goals).

If you do this for long enough, you’ll eventually find yourself in one of the staff engineer archetypes. However, it probably won’t be the one you’re “aiming for”. The whole point of being a staff engineer is that you’re willing to fill whatever archetype the company needs at the time.

Final thoughts

In his original staff engineer post, Larson is pretty clear that these archetypes are more of an anthropological description of some of the varied niches staff engineers fill, not a how-to guide for succeeding in the role⁵. At the time, the “staff engineer” role was fairly new and people were still trying to figure out what it even meant. Pointing out that there were a few very different ways to succeed in the role was a genuinely novel observation.

The staff engineer archetypes are a good list of ways an engineer can be very useful to their organization - but only once they’ve built a deep relationship of trust with their organization’s leadership. Advice on how to succeed as a staff engineer should be about **how to build that trust**, not about what to do once you have it.

1. One caveat that is too pedantic for the body of the post: each tech company has a different structure of roles. Some don’t have the formal “staff” title at all, while others have “staff” as a fairly early rung on the ladder and a panoply of “senior staff”, “senior principal staff”, and so on roles above it. Like all “staff engineer” discourse, this post is not about the word itself but about the point in the engineering job ladder where progression becomes significantly more difficult.

↩

2. Impressing your VP's trusted lieutenants can actually be a good way to build trust in the medium-term, but you'd better hope you've built enough understanding of the system to do it right. If this process goes badly, your reputation in the org might be torched for years.

↩

3. In theory, at least. In practice it's always better to be useful (again, in the sense of "delivering shareholder value").

↩

4. This is why very senior leadership sometimes seem so unempathetic towards engineering complaints: their work environment operates by very different rules and norms to that of most engineers. I keep meaning to try and write about this and never succeeding. This draft is the closest thing I have to a deeper exploration of the point.

↩

5. For the record, my how-to guides are here and here.

↩